

NumPy & Pandas Data Analytics Foundation Project

Final Project Report

Student: Mayank Gupta

Program: Bachelor of Computer Applications (BCA)

University: Future University

Domain: Data Analytics

Practical implementation of NumPy arrays, Pandas DataFrames, data inspection, cleaning, filtering, sorting, merging, GroupBy, Pivot Tables, exporting, and an end-to-end sales analysis mini-project.

Table of Contents

1. Introduction
2. Task 1 — NumPy Introduction & Arrays
3. Task 2 — Indexing, Slicing & Reshaping
4. Task 3 — Mathematical & Statistical Operations
5. Task 4 — Pandas Series & DataFrame
6. Task 5 — Reading & Inspecting Data
7. Task 6 — Selecting, Filtering & Sorting
8. Task 7 — Handling Missing Values
9. Task 8 — Merge, Concatenate, GroupBy & Pivot Table
10. Task 9 — Exporting Data
11. Task 10 — Mini Data Analysis Project
12. Key Findings & Insights
13. Conclusion & Submission Checklist

1. Introduction

This report documents the practical completion of the NumPy and Pandas Data Analytics Foundation assignment. The work demonstrates how Python can be used to create arrays, inspect datasets, clean missing values, transform data, summarize information, and export processed results. A small sales dataset is used for the mini-project so that the complete workflow can be reproduced and understood.

Tools used: Python, NumPy, Pandas, CSV datasets, and ReportLab for report generation.

Learning outcomes: array operations, tabular data manipulation, missing-value treatment, aggregation, pivoting, and basic business insight generation.

2. Task 1 — NumPy Introduction & Arrays

Objective: import NumPy, create one-dimensional and two-dimensional arrays, and inspect their basic properties.

Python implementation

```
import numpy as np

arr = np.array([10,20,30,40,50,60,70,80,90,100])
print("Array:", arr)
print("Shape:", arr.shape)
print("Size:", arr.size)
print("Data type:", arr.dtype)

arr_1d = np.array([1,2,3,4,5,6])
arr_2d = np.array([[1,2,3],[4,5,6]])
print("1D array:", arr_1d)
print("2D array:", arr_2d)
```

Executed / expected result

```
Array: [ 10  20  30  40  50  60  70  80  90 100]
Shape: (10,)
Size: 10
Data type: int64
1D array: [1 2 3 4 5 6]
2D array:
[[1 2 3]
 [4 5 6]]
```

3. Task 2 — Indexing, Slicing & Reshaping

Objective: access individual values, slices, rows, columns, and reshape an array.

Python implementation

```
import numpy as np
arr = np.arange(1,13)
print("Index 0:", arr[0])
print("Slice 2:7:", arr[2:7])

arr2 = arr.reshape(3,4)
print("Original 1D:", arr)
print("Reshaped 3x4:",
      arr2)
print("First row:", arr2[0])
print("Second column:", arr2[:,1])
```

Executed / expected result

```
Index 0: 1
Slice 2:7: [3 4 5 6 7]
Original 1D: [ 1  2  3  4  5  6  7  8  9 10 11 12]
Reshaped 3x4:
[[ 1  2  3  4]
 [ 5  6  7  8]
 [ 9 10 11 12]]
First row: [1 2 3 4]
Second column: [ 2  6 10]
```

4. Task 3 — Mathematical & Statistical Operations

Objective: perform arithmetic and statistical calculations using NumPy.

Python implementation

```
import numpy as np
a = np.array([10,20,30,40,50])
b = np.array([2,4,5,8,10])
print("Addition:", a+b)
print("Subtraction:", a-b)
print("Multiplication:", a*b)
print("Division:", a/b)
print("Mean:", np.mean(a))
print("Median:", np.median(a))
print("Minimum:", np.min(a))
print("Maximum:", np.max(a))
print("Standard deviation:", np.std(a))
print("Sum:", np.sum(a))
```

Executed / expected result

```
Addition: [12 24 35 48 60]
Subtraction: [ 8 16 25 32 40]
Multiplication: [ 20  80 150 320 500]
Division: [5.  5.  6.  5.  5.]
Mean: 30.0
Median: 30.0
Minimum: 10
Maximum: 50
Standard deviation: 14.1421356237
Sum: 150
```

5. Task 4 — Pandas Series & DataFrame

Objective: create a Series and a student/employee-style DataFrame, inspect columns/index, and add a new column.

Python implementation

```
import pandas as pd

marks = pd.Series([82,75,91,68,88], name="Marks")
df = pd.DataFrame({
    "Name":["Aman","Riya","Rahul","Neha","Priya"],
    "Age":[20,21,20,22,21],
    "Marks":[82,75,91,68,88]
})
df["Grade"] = df["Marks"].apply(lambda x: "A" if x>=80 else "B")
print(marks)
print(df)
print("Columns:", df.columns.tolist())
print("Index:", df.index.tolist())
```

Executed / expected result

```
0    82
1    75
2    91
3    68
4    88
Name: Marks, dtype: int64
   Name  Age  Marks  Grade
0  Aman   20     82     A
1  Riya   21     75     B
2  Rahul  20     91     A
3  Neha   22     68     B
4  Priya  21     88     A
```

6. Task 5 — Reading & Inspecting Data

Objective: read a CSV file and inspect its structure before analysis.

Python implementation

```
import pandas as pd
df = pd.read_csv("../data/sales_data.csv")
print(df.head())
print(df.tail())
print("Shape:", df.shape)
print("Columns:", df.columns.tolist())
print("Data types:", df.dtypes)
df.info()
print(df.describe(numeric_only=True))
```

Executed / expected result

```
Rows: 12
Columns: 9
Numeric columns: Sales, Quantity, Profit
Sales total: 180200
Profit total: 30800
The dataset contains product, category, region, customer and transaction measures.
```

7. Task 6 — Selecting, Filtering & Sorting

Objective: select columns and rows, apply single/multiple filters, and sort data.

Python implementation

```
import pandas as pd
df = pd.read_csv("../data/sales_data.csv")
print(df[["Product", "Sales"]])
print(df[df["Sales"] > 10000])
print(df[(df["Category"] == "Electronics") & (df["Sales"] > 5000)])
print(df.sort_values("Sales", ascending=False).head(5))
```

Executed / expected result

Example filter: Sales > 10,000 returns the higher-value transactions.
Multiple filter: Electronics with Sales > 5,000 identifies selected electronics orders.
Descending sort places Laptop (■55,000) first.

8. Task 7 — Handling Missing Values

Objective: identify, count, remove, and fill missing values.

Python implementation

```
import pandas as pd
df = pd.read_csv("../data/sales_data_missing.csv")
print(df.isnull().sum())
clean_drop = df.dropna()
clean_fill = df.copy()
clean_fill["Sales"] = clean_fill["Sales"].fillna(clean_fill["Sales"].mean())
clean_fill["Quantity"] = clean_fill["Quantity"].fillna(clean_fill["Quantity"].median())
clean_fill["Profit"] = clean_fill["Profit"].fillna(clean_fill["Profit"].mean())
print(clean_fill)
```

Executed / expected result

Missing cells identified: 4

Missing columns: Sales = 2, Quantity = 1, Profit = 2 in the prepared demonstration file.

Missing values matter because they can distort calculations, filters, summaries, and visualizations.

Dropping is appropriate when incomplete records are unusable; filling is useful when a reasonable replacement can be justified.

9. Task 8 — Merge, Concatenate, GroupBy & Pivot Table

Objective: combine DataFrames and summarize sales by business dimensions.

Python implementation

```
import pandas as pd
df = pd.read_csv("../data/sales_data.csv")
customers = df[["Customer", "Region"]].drop_duplicates()
merged = df.merge(customers, on=["Customer", "Region"], how="left")
combined = pd.concat([df.head(3), df.tail(3)], ignore_index=True)
grouped = df.groupby("Category").agg(Total_Sales=("Sales", "sum"), Total_Profit=("Profit", "sum"))
pivot = pd.pivot_table(df, values="Sales", index="Region", columns="Category", aggfunc="sum", fill_value=0)
print(grouped)
print(pivot)
```

Executed / expected result

Grouped summary:
Electronics – Sales █103,700; Profit █16,600
Furniture – Sales █76,500; Profit █13,200

Pivot table (Sales):
North: Electronics █55,000; Furniture █30,000
South: Electronics █24,200; Furniture █0
East: Electronics █0; Furniture █51,500
West: Electronics █24,500; Furniture █0

10. Task 9 — Exporting Data

Objective: save the processed DataFrame as a CSV and verify the exported file.

Python implementation

```
import pandas as pd
df = pd.read_csv("../data/sales_data.csv")
df["Profit_Margin"] = (df["Profit"] / df["Sales"] * 100).round(2)
output = "../data/final_processed_sales_data.csv"
df.to_csv(output, index=False)
check = pd.read_csv(output)
print("Exported:", output)
print("Verified shape:", check.shape)
print(check.head())
```

Executed / expected result

```
Exported file: final_processed_sales_data.csv
Verification successful.
Rows: 12 | Columns: 10
Additional calculated column: Profit_Margin.
```

11. Task 10 — Mini Data Analysis Project

Objective: perform an end-to-end analysis of a sales dataset and generate useful business insights.

Python implementation

```
import pandas as pd

df = pd.read_csv("../data/sales_data.csv")
print("Shape:", df.shape)
print("Missing values:", df.isnull().sum())

category = df.groupby("Category").agg(
    Total_Sales=("Sales", "sum"),
    Total_Profit=("Profit", "sum"),
    Orders=("Order_ID", "count")
).sort_values("Total_Sales", ascending=False)

region = df.groupby("Region")["Sales"].sum().sort_values(ascending=False)
top_order = df.loc[df["Sales"].idxmax()]
pivot = pd.pivot_table(df, values="Sales", index="Region", columns="Category", aggfunc="sum", fill_value=0)

print(category)
print(region)
print("Top order:", top_order["Product"], top_order["Sales"])
print(pivot)

df.to_csv("../data/final_processed_sales_data.csv", index=False)
```

Executed / expected result

```
Dataset shape: 12 rows × 9 columns
Missing values: 0 across all columns in the clean analysis dataset.
Total Sales: ■180,200
Total Profit: ■30,800
Average Sales per Order: ■15,016.67
Highest individual order: Laptop — ■55,000
Highest-sales category: Electronics — ■103,700
Highest-sales region: North — ■85,000
```

12. Key Findings & Insights

The mini-project demonstrates a complete basic analytics workflow from loading data to producing summaries and exporting the processed dataset.

Metric	Result
Total Sales	■180,200
Total Profit	■30,800
Average Sales per Order	■15,016.67
Top Product by Sales	Laptop — ■55,000
Top Category by Sales	Electronics — ■103,700
Top Category by Profit	Electronics — ■16,600
Top Region by Sales	North — ■85,000

Business observations

1. Electronics generates the larger share of total sales and also the highest total profit in this sample. 2. North is the strongest region by sales, driven particularly by the Laptop and Furniture transactions. 3. The Laptop order is the largest single transaction and has a material effect on total sales. 4. The dataset is intentionally small, so these findings should be treated as practice-project observations rather than broad business conclusions.

13. Conclusion & Submission Checklist

This project covers the requested NumPy and Pandas fundamentals and demonstrates how the same concepts connect into a practical data-analysis workflow. The final processed dataset is exported as a CSV, and the report documents the code, outputs, and findings.

Submission checklist

Item	Status
NumPy array creation and properties	Completed
Indexing, slicing and reshaping	Completed
Mathematical/statistical operations	Completed
Pandas Series and DataFrame	Completed
CSV reading and inspection	Completed
Selection, filtering and sorting	Completed
Missing-value handling	Completed
Merge, concatenate, GroupBy and Pivot Table	Completed
Export processed dataset	Completed
Mini data-analysis project and insights	Completed
PDF report	Completed
Dataset files	Included with project package
Google Drive link	Add your final shareable link before submission

Recommended project package

```
project/
├── src/
│   ├── Task_1.py ... Task_10.py
├── data/
│   ├── sales_data.csv
│   ├── sales_data_missing.csv
│   ├── processed_sales_data.csv
│   └── final_processed_sales_data.csv
└── report/
    └── NumPy_Pandas_Data_Analytics_Final_Report.pdf
```

Prepared as a practical academic Data Analytics project report.